Paltuu Feed Algorithm: Complete Technical Specification

1. Core Concept

Every user on Paltuu has a **preference vector** — a mathematical representation of what they care about. Every post has a **tag vector** — a representation of what it is about. The feed algorithm matches these two vectors and ranks posts accordingly. A new user's preference vector is empty, so they see popular content. As they interact, the vector fills in and the feed becomes personalized.

The system has three phases that a user moves through automatically based on their behaviour history.

2. The Category Taxonomy

Every post gets tagged from this list. Tags are applied by moderators (you, manually, at launch). Aim for 2–4 tags per post maximum — over-tagging dilutes the signal.

Species (pick one) `cat` `dog` `bird` `fish` `reptile` `rabbit` `hamster` `other-animal`

Breed (free tag, normalised) `persian` `siamese` `golden-retriever` `labrador` `german-shepherd` etc. Add breeds as they appear. These are optional and more specific than species.

Owner Identity (pick one) `new-pet-owner` `experienced-owner` `adopter` `foster-parent` `breeder` `rescuer` `multi-pet-owner`

Content Type (pick one) `funny-content` `heartwarming` `educational` `product-review` `adoption-story` `rescue-story` `milestone` `health-updatequestion` `rant` `general`

Topic (pick up to two) `grooming` `nutrition` `training` `health-vet` `behaviour` `lost-found` `product-recommendation` `event` `community`

Tone `positive` `urgent` `informational` `humorous`

This gives you roughly 50–60 meaningful category dimensions. Every user builds a weight for each one.

3. The User Preference Vector

Stored in the `user_interest_scores` table. One row per user per category.

user_id	category	score	event_count	last_updated
42	cat	0.84	47	2026-04-30
42	funny-content	0.71	31	2026-04-30
42	health-vet	0.22	4	2026-04-28

The score is always between 0.0 and 1.0. It is not a raw count — it is a **decayed weighted average** that we will define precisely in Section 5.

4. The Engagement Events and Their Raw Weights

These are the actions a user can take on Paltuu and the raw signal weight each one carries. These numbers are starting points — they should be tuned after 4–6 weeks of real data.

Positive Events

Event	Raw Weight	Rationale
Paw (like)	1.0	Baseline signal. Low effort, moderately meaningful.
View duration 3–10 seconds	0.5	Paused and read. Passive but real interest.
View duration 10–30 seconds	1.2	Read carefully. Stronger than a like.
View duration 30+ seconds	2.0	Deeply engaged. Very strong signal.
Opened comment section (did not comment)	1.5	Curious enough to check discussion.
Posted a comment	3.5	High-effort positive action. Strong signal.
Replied to a comment on this post	3.0	Engaged in conversation. Strong.
Reposted	4.5	Strongest distribution signal. Wants others to see it.
Tapped through to poster's profile	2.5	Curious about the person, not just the post.
Followed poster from their profile	5.0	Maximum positive signal on a single post.
Used native share	3.0	Wanted to distribute outside the app.
Tapped on a tag to explore it	1.5	Explicit interest signal in that category.

Negative Events

Event	Raw Weight	Rationale
Scroll past in under 1 second	-0.5	Mild negative — could be timing.
Scroll past in under 0.3 seconds	-1.0	Actively skipped. Clearer negative.
Tapped "Not Interested"	-4.0	Explicit rejection. Very strong negative.

Reported post	-5.0	Strongest negative. Remove from this user's feed immediately.
Blocked poster	-6.0	Nuclear option. Never show this poster again.

Why negatives are smaller in count but higher in per-event impact: Users do not tap "Not Interested" often. When they do, it means something. A user might scroll past 200 posts casually and like 10 — the passive scrolls are weak signal. The one "Not Interested" tap is a very strong signal. The weights reflect this asymmetry.

5. Updating the Preference Vector

When a user performs an engagement event on a post tagged with category C, update that category's score using a **decayed exponential moving average**:

new_score = (α × event_weight) + ((1 - α) × old_score)

Where α (alpha) is the **learning rate** — how much each new event shifts the score. Start with **α = 0.15**.

This means:

- Old behaviour is remembered but slowly fades
- Recent behaviour has more influence than old behaviour
- A single extreme event cannot completely override a long history

Example:

The user has a cat score of 0.60. They repost a cat post (raw weight 4.5, normalised to 1.0 since scores are capped at 1.0):

new_score = (0.15 × 1.0) + (0.85 × 0.60) = 0.15 + 0.51 = 0.66

User then taps "Not Interested" on a cat post (raw weight -4.0, normalised to -1.0):

new_score = (0.15 × -1.0) + (0.85 × 0.66) = -0.15 + 0.561 = 0.411

The score drops significantly but does not go to zero from one negative event.

Normalising raw weights to the -1.0 to 1.0 range:

normalised_weight = raw_weight / max_possible_raw_weight

Max positive raw weight is 5.0 (follow). Max negative is -6.0 (block). Normalise accordingly before applying to the score update. Scores are clamped to [0.0, 1.0] after update. Negative scores floor at 0.

Time decay on scores:

Run a nightly batch job that applies passive decay to all scores that have not been updated in the last 7 days:

decayed_score = current_score × (decay_rate ^ days_since_last_event)

Use `decay_rate = 0.98` per day. A score of 0.80 with no activity for 30 days becomes:

0.80 × (0.98 ^ 30) = 0.80 × 0.545 = 0.436

This ensures that a user who abandons cats for a month and starts posting about dogs sees their feed adjust naturally.

6. The Three User Maturity Phases

This answers your question about when to stop showing popular content and start showing personalised content.

Phase 0 — New User (Cold Start)

Trigger: User has fewer than 30 total engagement events recorded.

Feed composition:

- 70% Trending pool (top posts platform-wide by engagement score, last 48 hours)
- 20% Posts from any accounts they follow (if any)
- 10% Random sample from all recent posts (last 24 hours)

Goal: Gather first signals. Learn species preference fast by noting which trending posts they engage with.

Onboarding accelerator: During signup, ask two questions: "What animals do you have or love?" and "What kind of content do you prefer?" Use answers to pre-seed the preference vector with starting scores of 0.3 for selected categories. This is not the same as earning 0.3 — it is a soft prior that gets overridden quickly by real behaviour. Better than starting at zero.

Phase 1 — Emerging (Learning)

Trigger: 30–200 total engagement events.

Feed composition:

- 40% Trending pool (now filtered to trending posts that match user's top 3 categories)
- 35% Interest-based posts (posts tagged with categories where user score > 0.3)
- 25% Posts from followed accounts

Goal: Validate emerging interests and grow the follow graph. At this phase, insert follow suggestions in-feed every 8th post — accounts that post frequently in the user's top categories.

Phase 2 — Mature (Personalised)

Trigger: 200+ total engagement events and following 10+ accounts.

Feed composition:

- 60% Posts from followed accounts (ranked by post score, not raw chronology)
- 30% Interest-based posts from non-followed accounts

- 10% Discovery (intentional exposure to categories with low or zero score, to enable new interests)

Goal: Deliver a high-quality personal feed while keeping 10% discovery so the feed never becomes a closed loop.

The 10% discovery slot is important. Without it, a user who only ever sees cat content will never discover dog content even if they might love it. The discovery slot deliberately injects variety. If the user engages with discovery content, their vector updates and that category grows. If they consistently skip discovery content, reduce its weight gradually.

7. The Post Scoring Function

When building a user's candidate feed, every candidate post gets a score. Posts are sorted by this score and the top N are served.

```
post_score =
  (interest_match    × 0.35) +
  (recency           × 0.20) +
  (engagement_score  × 0.25) +
  (relationship      × 0.20)
```

Interest Match (0.0 to 1.0)

Average of the user's preference scores for all tags on this post.

interest_match = mean(user_score[tag] for tag in post.tags)

If a post is tagged `cat` and `funny-content` and the user has scores of 0.80 and 0.65:

interest_match = (0.80 + 0.65) / 2 = 0.725

Recency Score (0.0 to 1.0)

Exponential decay from post creation time. Half-life of 12 hours for Phase 0/1 users (trending matters more), 24 hours for Phase 2 users (follow graph matters more, people post less frequently).

recency = e ^ (-λ × hours_since_post)

Where λ = ln(2) / half_life_hours. For 12-hour half-life: λ = 0.0578.

A post 6 hours old scores ~0.71. A post 24 hours old scores ~0.25. A post 72 hours old scores ~0.02.

Engagement Score (0.0 to 1.0)

Platform-wide engagement quality, normalised against the poster's typical engagement rate. This prevents large accounts from always dominating and gives small accounts with highly engaged posts a fair chance.

raw_engagement = likes + (comments × 5) + (reposts × 8) + (shares × 3)
normalised = raw_engagement / (1 + raw_engagement) ← squash to 0-1 range
engagement_score = normalised × creator_quality_factor

Creator quality factor: ratio of this post's engagement to the poster's average post engagement. A post performing 3x above the account's average gets a boost. This rewards quality over follower count.

Relationship Score (0.0 to 1.0)

0.0 = no relationship
0.3 = follow exists
0.5 = follow + prior engagement (liked/commented on their posts before)
0.8 = follow + regular engagement + recent interaction (last 7 days)
1.0 = mutually follow + high engagement history

8. The Feed Assembly Algorithm

Run this when a user requests their feed (on app open or on scroll-to-refresh):

1. Determine user phase (Phase 0, 1, or 2) from event_count

2. Pull candidate pool:
   - Phase 0: top 200 trending posts (last 48h) + all followed-account posts (last 7d)
   - Phase 1: top 100 trending in top-3 categories + interest posts + followed posts
   - Phase 2: followed-account posts (last 7d) + interest posts (last 3d) + 10% discovery

3. Remove already-seen posts (check post_impressions table)
4. Remove posts from blocked users
5. Remove posts reported by this user
6. Score every remaining candidate using the formula in Section 7
7. Sort by score descending
8. Apply diversity pass:
   - No more than 2 consecutive posts from the same account
   - No more than 3 consecutive posts from the same species tag
   - Every 8th post in Phase 0/1: insert a follow suggestion card
9. Return top 20 posts as the next page
10. Log impressions for all 20 posts in post_impressions table

Pagination: When the user scrolls to the bottom, repeat steps 1–9 with the already-seen pool updated. Do not cache the full feed — regenerate each page fresh so new posts and new engagement signals are reflected immediately.

9. Database Tables Needed

Beyond what was already specified in the schema discussion:

```
-- User preference vector
CREATE TABLE user_interest_scores (
  user_id         integer REFERENCES users(user_id) ON DELETE CASCADE,
  category        text NOT NULL,
  score           numeric DEFAULT 0 CHECK (score >= 0 AND score <= 1),
  event_count     integer DEFAULT 0,
  last_updated    timestamptz DEFAULT now(),
  PRIMARY KEY (user_id, category)
);
```

```
-- Post tags (applied by moderators at launch)
CREATE TABLE social_post_tags (
  post_id       bigint REFERENCES social_posts(post_id) ON DELETE CASCADE,
  tag           text NOT NULL,
  tag_type      text NOT NULL CHECK (tag_type IN
                ('species','breed','owner_type','content_type','topic','tone')),
  PRIMARY KEY (post_id, tag)
);

-- All engagement events (the raw event log — never aggregate here, keep raw)
CREATE TABLE feed_events (
  event_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id       integer REFERENCES users(user_id) ON DELETE CASCADE,
  post_id       bigint REFERENCES social_posts(post_id) ON DELETE CASCADE,
  event_type    text NOT NULL,   -- 'like','comment','repost','view_3s','view_10s',
                                 -- 'view_30s','open_comments','follow_from_post',
                                 -- 'profile_tap','share','not_interested','report'
  raw_weight    numeric NOT NULL,
  created_at    timestamptz DEFAULT now()
);

CREATE INDEX idx_feed_events_user ON feed_events(user_id, created_at DESC);
CREATE INDEX idx_feed_events_post ON feed_events(post_id);

-- Post impressions (what was shown to whom)
CREATE TABLE post_impressions (
  impression_id     bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  post_id           bigint REFERENCES social_posts(post_id) ON DELETE CASCADE,
  user_id           integer REFERENCES users(user_id) ON DELETE CASCADE,
  view_duration_ms  integer,
  phase_at_time     smallint,   -- which phase the user was in (0, 1, 2)
  feed_score        numeric,    -- the score the post received when served
  source            text,       -- 'followed','interest','trending','discovery'
  viewed_at         timestamptz DEFAULT now()
);

CREATE INDEX idx_impressions_user_recent ON post_impressions(user_id, viewed_at DESC);

-- User maturity tracking (denormalised for speed)
ALTER TABLE users
  ADD COLUMN IF NOT EXISTS feed_phase       smallint DEFAULT 0,
  ADD COLUMN IF NOT EXISTS total_feed_events integer DEFAULT 0,
  ADD COLUMN IF NOT EXISTS feed_following_count integer DEFAULT 0;
```

10. View Duration Tracking — Client Implementation

This is where most teams make mistakes. You cannot track view duration by sending an event to the server on every scroll. That would generate thousands of API calls per session and drain the user's battery.

The correct approach:

On the client (React Native), use a visibility tracker on each post in the FlatList. When a post enters the viewport, start a timer. When it leaves, stop the timer and add the duration to a local batch array. When the user backgrounds the app or navigates away, flush the batch to the server in a single API call.

```
// Pseudocode for the batch flush
const impressionBatch: ImpressionEvent[] = [];

onPostVisible(postId: string) {
  impressionBatch.push({ postId, startTime: Date.now() });
}

onPostHidden(postId: string) {
  const entry = impressionBatch.find(e => e.postId === postId && !e.endTime);
  if (entry) entry.endTime = Date.now();
}

onAppBackground() {
  const completed = impressionBatch
    .filter(e => e.endTime)
    .map(e => ({ postId: e.postId, durationMs: e.endTime - e.startTime }));

  api.post('/api/social/impressions/batch', { events: completed });
  impressionBatch = [];
}
```

The server endpoint processes the batch, determines which duration bucket each event falls into (under 1s, 1–3s, 3–10s, 10–30s, 30s+), writes to `feed_events`, and updates `user_interest_scores` asynchronously via a queue.

11. The Moderator Tagging Workflow

At launch, every post that goes live needs to be tagged before it can be fed into the algorithm. Since volume is low at launch, this is manageable manually.

Tagging queue: Add an `is_tagged` boolean to `social_posts`. Default is false. A simple admin page shows all untagged posts. Moderator sees the post, taps applicable tags, hits save. `is_tagged` flips to true and the post enters the feed pool.

Tag suggestion (Phase 2 of moderation): Once you have 500+ tagged posts, you can use a simple keyword-match model to suggest tags based on post content and let the moderator approve or adjust rather than starting from scratch. This is not ML — it is just regex and keyword lists. "My cat" → suggest `cat`. "Just adopted" → suggest `adopter` + `adoption-story`. Cuts tagging time by 80%.

Auto-tagging from pet profile: If a post has a pet attached (`pet_id` is not null), auto-apply the species and breed tags from the pet's profile. This covers the most common case automatically.

12. Tuning Guide — What to Watch After Launch

Run these checks at the 2-week, 4-week, and 8-week marks:

Feed health signals (good):

- Average session length increasing week over week
- Percentage of users returning day 2, day 7, day 30 stable or growing
- Average engagement rate (likes + comments / impressions) above 3%
- Users moving from Phase 0 to Phase 1 within first 3 sessions

Feed health signals (bad):

- Users opening the app and closing without engaging (< 5 seconds session) more than 30% of opens — feed is not relevant
- "Not Interested" rate above 5% of impressions — categories are wrong or weights are off
- Phase 0 users staying in Phase 0 after 7 days — they are not engaging enough to build a signal; the onboarding question might not be converting to useful pre-seeds

First tuning levers:

- If feed feels too repetitive: increase the discovery allocation from 10% to 20%
- If feed feels too random: decrease alpha from 0.15 to 0.08 (slower learning, more weight to history)
- If follow suggestions are being ignored: change the trigger from every 8th post to every 5th, and only show accounts with 5+ posts
- If small creators are invisible: increase the creator quality factor weight relative to raw engagement score

13. What Umer Needs to Build

Backend (Next.js API routes):

POST /api/social/feed Returns next page of scored posts for authenticated user
POST /api/social/impressions/batch Receives batched view duration events from client
POST /api/social/event Receives explicit events (like, comment, repost, not-interested, report)
GET /api/social/feed/phase Returns current phase and event count for the user (for debugging)

The `/feed` endpoint runs the assembly algorithm from Section 8. It should respond in under 300ms. If scoring 200 candidates takes longer, precompute scores every 15 minutes in a background job and cache them per user in Redis or a simple `feed_cache` table.

Admin (moderator tagging):

GET /api/admin/social/untagged Returns posts pending tagging
POST /api/admin/social/tag/:id Applies tags to a post and sets is_tagged = true

Client (React Native):

- Visibility tracker on FlatList items using `onViewableItemsChanged`
- Batch impression flush on app background using `AppState` listener
- Explicit event firing on like, comment open, repost, profile tap, share, not-interested, report
- Feed phase indicator visible in dev mode so you can see which phase a test account is in